

We re-implemented four canonical deep RL algorithms from scratch in PyTorch and benchmarked them on four classic-control Gymnasium environments with three seeds each. The algorithms span the two main axes of the deep RL taxonomy:

- value-based vs. policy-based,
- on-policy vs. off-policy.

We report bootstrap 95% confidence intervals over seeds, pre-register four ablations, and document the implementation details that distinguish a paper recipe from a working agent.

DQN [?] (value-based, off-policy, discrete):

$$\mathcal{L}_{\text{DQN}} = \mathbb{E} \left[\ell_{\text{Huber}} \left(Q_{\theta}(s, a) - \left(r + \gamma (1-d) \max_{a'} Q_{\bar{\theta}}(s', a') \right) \right) \right].$$

ε -greedy exploration, target network synced every τ_{sync} steps, Huber loss for robustness.

PPO [?] (policy-based, on-policy, discrete + continuous):

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E} \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t) \right].$$

Clipped surrogate as a first-order trust-region; GAE advantages with (γ, λ) .

SAC [?] (policy-based, off-policy, continuous):

$$J_{\text{SAC}}(\pi) = \mathbb{E}_{\pi} \left[\sum_t \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}[\pi(\cdot|s_t)]) \right].$$

Squashed-Gaussian actor with tanh-Jacobian log-prob correction; auto-tuned α via log α .

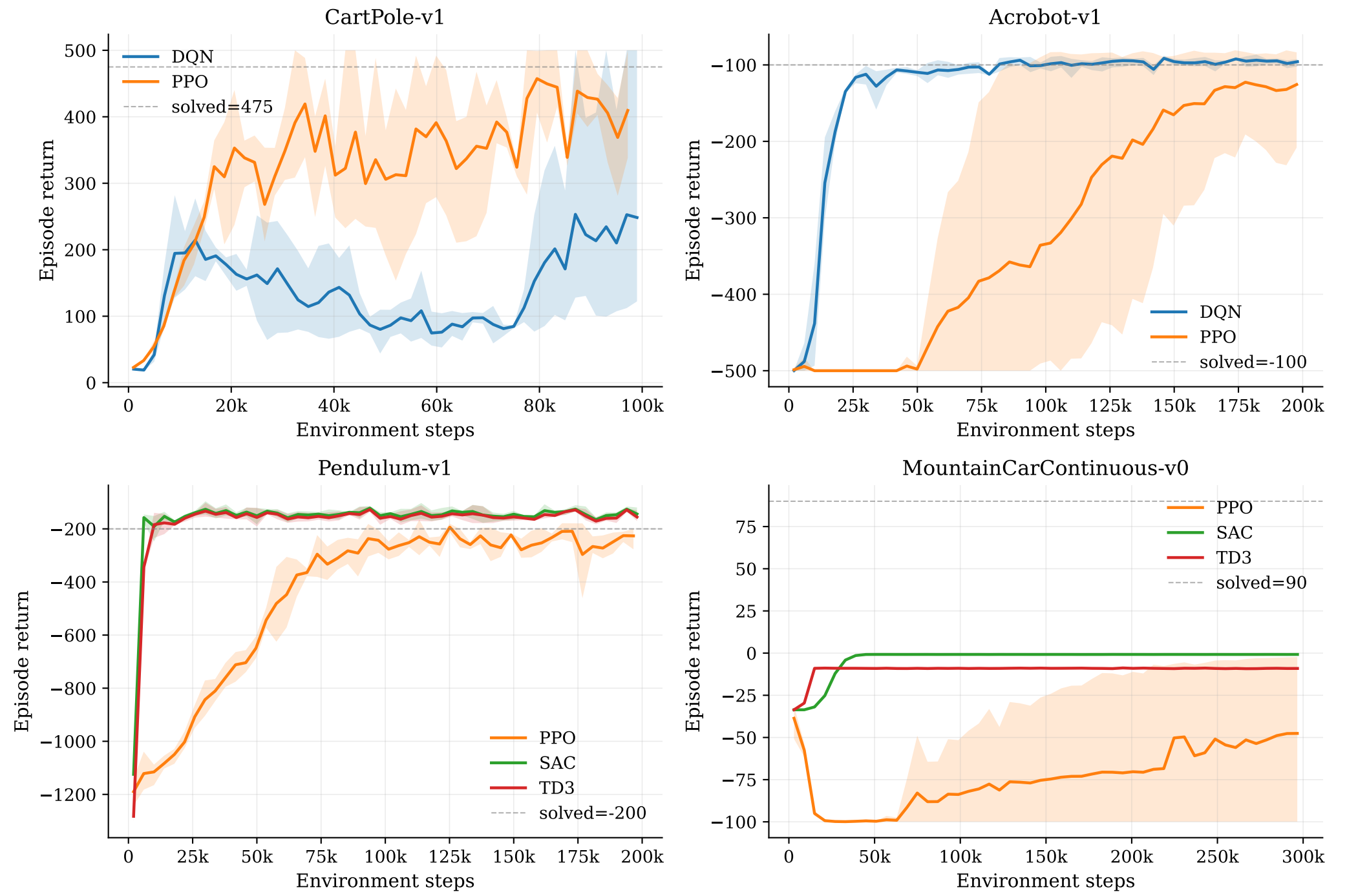
TD3 [?] (policy-based, off-policy, continuous):

$$y = r + \gamma (1-d) \min_{i \in \{1,2\}} Q_{\bar{\theta}_i}(s', \text{clip}(\pi_{\bar{\theta}}(s') + \tilde{\epsilon}, a_{\min}, a_{\max})).$$

Twin Q-targets, target policy smoothing ($\tilde{\sigma}=0.2$), delayed actor update every 2 critic steps.

Env	Action	Reward / budget
CartPole-v1	Discrete(2)	dense +1/step, 100k steps
Acrobot-v1	Discrete(3)	dense −1/step, 200k steps
Pendulum-v1	Box[−2, 2]	dense quadratic, 200k steps
MountainCarContinuous-v0	Box[−1, 1]	sparse +100 at goal , 300k steps

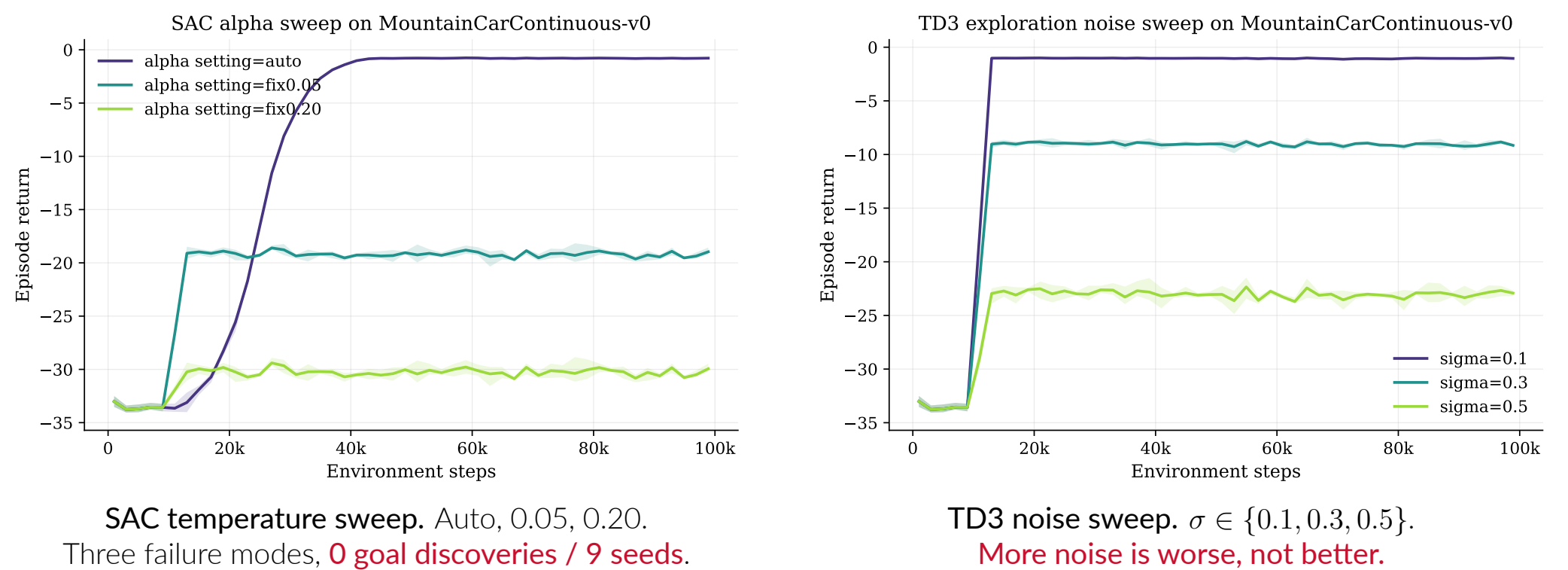
Protocol. 3 seeds $\{0, 1, 2\}$ per (algo, env). 95% bootstrap CI over seeds at every step bin. Networks: (64, 64) for discrete (DQN, PPO), (256, 256) for continuous (SAC, TD3), matching canonical papers. Hardware: RTX 3070 Laptop and EPFL gnoto V100. Code: PyTorch from scratch, no **stable-baselines3**.



Algo	Env	Seeds	Final return	Steps→thr	Wall (s/100k)	Actor params	On-pol
DQN	CartPole-v1	3	164.4 [111, 259]	100k	98	4.6k	no
DQN	Acrobot-v1	3	-96.1 [−101, −94]	30k	120	4.8k	no
PPO	CartPole-v1	3	380.1 [350, 402]	n/r	442	4.6k	yes
PPO	Acrobot-v1	3	-127.4 [−216, −82]	110k	457	4.8k	yes
PPO	Pendulum-v1	3	-247.9 [−265, −226]	95k	450	4.5k	yes
PPO	MountainCarContinuous-v0	3	-56.2 [−100, −5]	n/r	1961	4.4k	yes
SAC	Pendulum-v1	3	-146.1 [−152, −137]	7k	1183	67.3k	no
SAC	MountainCarContinuous-v0	3	-0.8 [−1, −1]	n/r	2713	67.1k	no
TD3	Pendulum-v1	3	-154.9 [−160, −148]	9k	483	67.1k	no
TD3	MountainCarContinuous-v0	3	-9.1 [−9, −9]	n/r	862	66.8k	no

- **CartPole.** **DQN bimodal** (1/3 seeds solve); PPO reliable (380, CI [350, 402]).
- **Acrobot.** DQN solves in $\sim 30k$ steps; PPO slower, one seed collapses.
- **Pendulum.** **SAC and TD3 are $\sim 13\times$ more sample-efficient than PPO** (SAC reaches threshold in 7.4k env steps vs PPO 95k).
- **MountainCarContinuous.** **Only PPO ever discovers the +100 goal** (1/3 seeds). SAC (−0.8) and TD3 (−9.1) plateau at a lazy local optimum.

Two pre-registered ablations on MountainCarContinuous test whether the available exploration knobs rescue SAC or TD3. **Neither does.**



Mechanism. Both off-policy mechanisms (entropy regularization, additive Gaussian noise) drive uniform coverage of the *local* action set, not directed exploration in state space. PPO's on-policy rollouts occasionally produce a long correlated action sequence that crosses the hill; off-policy replay samples are decorrelated and cannot.

Other ablations confirm the diary. DQN target-sync sweep is a U-shape ($\tau \in \{100, 1000, 5000\}$), explaining CartPole bimodality: $\tau=1000$ sits at the edge of stability. PPO clip ratio: 0.1 is too conservative; 0.2 and 0.3 are tied.

- **Most expensive per iteration:** SAC (1183 to 2713 s per 100k steps) > TD3 > PPO > DQN (98 to 120 s).
- **Most compact policy:** DQN & PPO ($\sim 4.6k$ actor params) vs SAC & TD3 ($\sim 67k$). Caveat: this gap follows our (64, 64) vs (256, 256) width choice, not the algorithms.
- **Best on continuous actions: no single winner.** SAC/TD3 dominate dense reward (Pendulum); PPO is the only one to solve sparse reward (MCC).
- **Best off-policy data efficiency:** SAC, by 12.9 $\times$ over PPO on Pendulum (7.4k vs 95k steps to threshold).